

Programmation C/C++ sou GNU Linux

Make et Mikefile

Compilation d'un projet C ou C++

Supposer qu'on dispose d'un projet :

main.c, structure.c, et operation.c

Il est possible de compiler ce projet de trois manières différentes

1^{ère} méthode: une seule commande

```
gcc -o prog main.c structure.c operation.c
```

2^{ème} méthode: une commande pour chaque fichier

```
gcc -c main.c
```

```
gcc -c structure.c
```

```
gcc -c operation.c
```

PROBLÉMATIQUE ET OBJECTIFS

```
gcc -o prog main.c structure.c  
operation.c
```



```
gcc -c main.c  
gcc -c structure.c  
gcc -c -Wall operation.c
```



Fichiers objects: main.o, structure.o et operation.o.

Pour créer l'exécutable qui regroupe les 3 fichiers → Edition des liens

```
gcc -o prog main.o structure.o operation.o
```

Question: Si je modifie main.c, que dois je faire?

Make et Makefile

Question: Si je modifie main.c, que dois je faire? → reCompiler le fichier modifié

```
gcc -c main.c  
→ Ensuite régénérer le fichier exécutable  
gcc -o prog main.o structure.o operation.o
```

3^{ème} méthode: Utiliser l'outil **Makefile**

→ son rôle est de produire automatiquement la séquence de commandes permettant de construire un projet **basé sur la construction d'un graphe les dépendances**

Principe

- main.c **produit** main.o
- main.o, structure.o et operation.o **permettent de construire le projet prog.**

→ Il suffit de décrire ces relations à Make

**produit : source
commande**

Make et Makefile

- Il faut indiquer à make laquelle construire en priorité : par convention, c'est la **première**.
- **la construction du projet au début du fichier makefile d'abord.**

Exemple

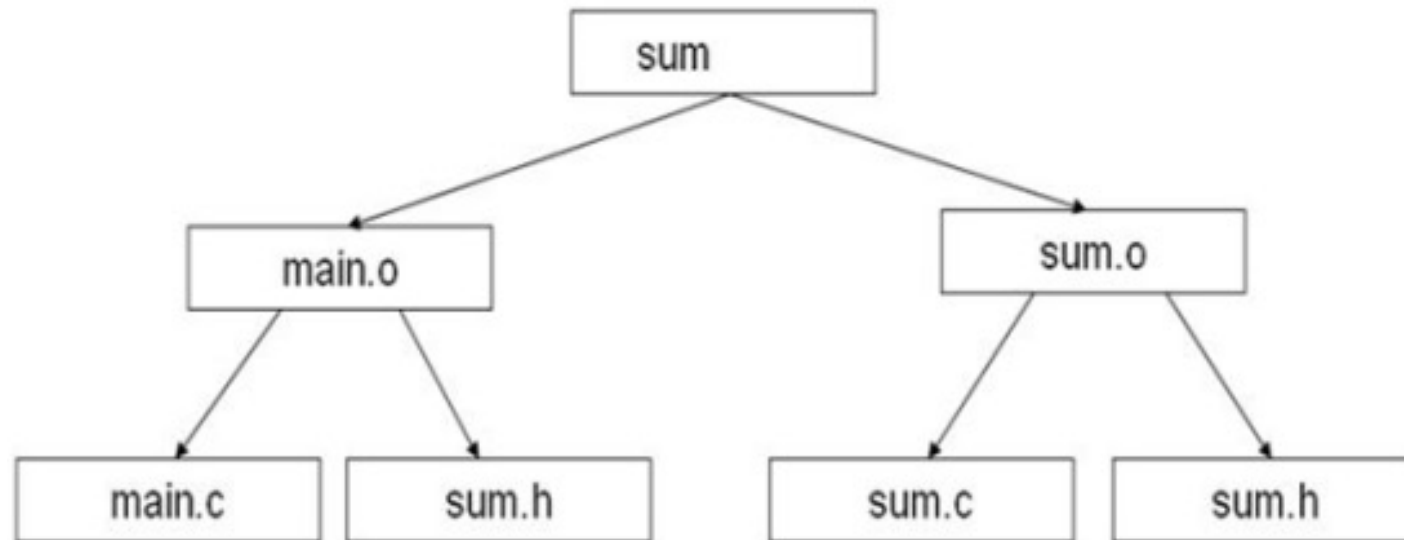
```
prog: main.o structure.o operation.o
    gcc -o prog main.o structure.o operation.o
main.o : main.c
    gcc -c main.c
structure.o : structure.c
    gcc -c structure.c
operation.o : operation.c
    gcc -c operation.c
```

Lors d'une modification, make se base sur la date de modification des fichiers pour déterminer les mises à jours à effectuer. `

Si fichier source modifié--> date de MAJ sup à celle du fichier .O → re-exécution automatique

*Initialement les fichiers .o n'existent pas ! → **compilation conditionnelle***

Make et Makefile: exercice 1



Contenu de Makefile?

Make et Makefile: solution

Contenu de Makefile?

```
sum: main.o sum.o
    gcc -o sum main.o sum.o
main.o: main.c sum.h
    gcc -c main.c
sum.o: sum.c sum.h
    gcc -c sum.c
```

Make et Makefile: exercice 2

Exemple de Makefile → interprétez?

```
all : executable
executable : file1.o file2.o
    gcc -o executable file1.o file2.o
file1.o : file1.c file1.h
    gcc -c file1.c
file2.o : file2.c file1.h file2.h
    gcc -c file2.c
clean :
    rm file1.o file2.o executable core
```


Exécution de Makefile

→ Exécution nom de fichier par défaut Makefile

```
$make all
```

La commande « make » interprète le fichier makefile et exécute les commandes contenues dans la règle « all » si les dépendances sont ok.

Supposer qu'on dispose d'un fichier contenu les actions à faire :

```
$make -f file
```

Make et Makefile

Effacer les fichiers temporaires

Pour effacer les fichiers objets du projet → Ecrire la règle suivante qui s'appelle clean par convention :

```
clean :  
    rm -f prog *.o
```

Archiver les fichiers Sources avec **Make**

```
zip:  
    tar -zcvf prog.tar.gz main.c structure.c operation.c Makefile
```

Fournir plusieurs versions d'un seul projet

```
prog: main.o structure.o operation.o  
    gcc -o prog main.o structure.o operation.o -lm  
main.o : main.c  
    gcc -c main.c  
...
```

Makefile avancé

1. Variables

Un nom de variable est une suite de caractères ne contenant pas `:`, `#` et `=`

```
CC= gcc
prog: main.o structure.o operation.o
$(CC) -o main.o structure.o operation.o
```

Ou

```
CC= gcc
objets= main.o structure.o operation.o
prog: $(objets)
$(CC) -o prog $(objets)
```

Makefile avancé

2. Utilisation des macros / macro-commandes

Variable/Macro-commande	Rôle
CC	Compilateur du langage C (cc ou gcc)
CXX ou CPP	Compilateur du langage C++ (c++ ou g++)
CFLAGS	Paramètres (ou options) à passer au compilateur du langage C
CXXFLAGS	Paramètres (ou options) à passer au compilateur du langage C++
LDFLAGS	Paramètres (ou options) de l'édition des liens
EXEC	Nom de l'exécutable à générer
RM	Commande utilisée pour effacer un fichier
\$@	Nom de la cible
\$<	Nom de la première dépendance
\$^	Liste des dépendances
\$?	Liste des dépendances plus récentes que la cible

Makefile avancé

→Exemples

La variable `$@` représente le produit (ou le but) d'une règle, en reprenant l'exemple précédent :

```
CC= gcc
objets= blob.o tga.o tri_blob.o
blob: $(objets)
$(CC) -o $@ $(objets) -lm
```

La variable `^` représente toutes les dépendances d'une règle, par exemple tous les objets lors de la création du projet :

```
CC= gcc
objets= blob.o tga.o tri_blob.o
blob: $(objets)
$(CC) -o $@ ^ -lm
```

Makefile avancé

→Exemples

La variable \$< représente la première dépendance, on peut l'utiliser pour compiler un source C :

```
CC= gcc
blob.o: blob.c
$(CC) -o $@ -c $<
```

Il possible, avec une seule règle implicite, de compiler tous les sources C :

```
% .o: %.c
$(CC) -o $@ -c $<
```

Il est possible de compiler un projet quelconque avec un Makefile de quelques lignes, par exemple prog :

```
CC= gcc          # compilateur
CFLAGS= -Wall    # options de compilation pour les sources C

objets= main.o structure.o operateur.o
blob: $(objets)
$(CC) -o $@ $^ -lm

%.o: %.c
$(CC) $(CFLAGS) -o $@ -c $<
```

Makefile avancé

3. Génération automatique des noms de fichiers

Générer automatiquement les noms des fichiers objets à partir des noms des fichiers sources, en remplaçant l'extension .c par .o :

```
SRC= main.c structure.c operation.c

# nommage automatique des fichiers objets d'apres les noms des sources C
OBJ= $(SRC:.c=.o)
```

→ \$(OBJ) contiendra main.o structure.o operation.o

Makefile avancé

3. Génération automatique des noms de fichiers

Exemple du makefile précédent avec nommage automatique des objets a partir des fichiers sources .c :

```
CC= gcc          # compilateur
CFLAGS= -Wall    # options de compilation pour les sources C

sources= main.c structure.c operation.c
objets= $(sources:.c=.o)

blob: $(objets)
    $(CC) -o $@ $^ -lm

%.o: %.c
    $(CC) $(CFLAGS) -o $@ -c $<
```


Bibliothèques statiques et partagées

1. Bibliothèques statiques

Le nombre de fichiers .o à lier ensemble peut devenir considérable, —> Difficile de déterminer ceux qui sont réellement utilisés.

=> La solution est **de regrouper plusieurs .o** dans un fichier unique, une bibliothèque (une archive). La commande pour ce faire est nommée ar :

```
$ ar -cr libtest.a test1.o test2.o
```

Par convention, le nom de la bibliothèque commence par **lib** et l'**extension de la bibliothèque est .a**

- Libtest.a est un archive qui regroupe test1.o et test2.o
- -c: créer une archive si elle n'existe pas déjà sinon elle est mise à jour
- -r: remplacer ou d'ajouter des fichiers à l'archive sans afficher de message d'avertissement

Bibliothèques statiques et partagées

2. Bibliothèques statiques

Retenez

Une bibliothèque a un emplacement visible :

1. `/usr/local/lib` si la librairie est susceptible d'être utilisée par plusieurs utilisateurs ;
2. La variable « `LD_LIBRARY_PATH` » pour les bibliothèques dynamiques
3. `~/lib` si la librairie est susceptible d'être utilisée par un seul utilisateur

Vous trouverez dans le répertoire `/usr/lib` :

- `libc.a` est la librairie standard C (fonctions `malloc`, `exit`, etc.);
- `libm.a` est la librairie mathématique (fonctions `sqrt`, `cos`, etc.);

Vous pouvez utiliser:

```
$ gcc -o prog main.o -L/home/user/lib -lprog
```

-L/home/user/lib : Cet indicateur -L indique à gcc d'ajouter `/home/user/lib` au chemin de recherche des bibliothèques

-lprog permet de chercher dans les répertoires par défaut la bibliothèque `libprog.a` ou `libprog.so`

Bibliothèques statiques et partagées

1. Bibliothèques partagées / dynamiques

- Une bibliothèque dynamique est liée avec le programme **au moment de l'exécution**, plutôt qu'au **moment de la compilation**.
- Le binaire final de l'application ne contient pas le code de la bibliothèque dynamique lui-même, mais plutôt des références vers les fonctions de la bibliothèque (contrairement au bibliothèque statique, l'exécutable est indépendant)
- Lors de l'exécution, les bibliothèques partagées sont chargées

```
% gcc -shared -fPIC -o libtest.so test1.o test2.o
```

-shared : créer une bibliothèque partagée (.so)

-fPIC: indique au compilateur de générer un code qui peut être exécuté indépendamment de son adresse de chargement dans la mémoire

-o: spécifier l'output

-wall: activer tous les avertissements (all warning)